

AI Report

We predict this text is

Human Generated

AI Probability

45%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

LAPORAN TUGAS 1 SO.docx - 10/19/2025

LAPORAN TUGAS 1

Disusun oleh:

Nama: Cynthia Citra Azaria

NIM: 2410651025

Mata Kuliah:

Sistem Operasi

Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB I PENDAHULUAN

1.1 Latar Belakang

Program ini bertujuan memberikan pemahaman dasar tentang proses di Linux, termasuk pembuatan child process, komunikasi antar-proses (IPC), sinkronisasi, dan penanganan sinyal. Dengan mengerjakan tugas ini, mahasiswa belajar:

Membuat proses baru menggunakan `fork()`.

Menyinkronisasi parent dan child process menggunakan `waitpid()`.

Mengirim data hasil komputasi child ke parent menggunakan `pipe()`.

Mengukur waktu eksekusi tiap proses dengan `gettimeofday()`.

Menangani interupsi (Ctrl+C) secara terkontrol menggunakan `SIGINT`.

1.2 Tujuan

Membuat 3 child process yang menjalankan tugas berbeda.

Memahami komunikasi antar-proses tanpa menggunakan library high-level.

Mempelajari pengukuran waktu eksekusi proses.

Menangani terminasi proses secara aman (graceful termination).

Menghindari zombie process melalui penggunaan `waitpid()`.

BAB II TINJAUAN PUSTAKA

Konsep

Penjelasan

`fork()`

Membuat child process.

`pipe()`

Jalur komunikasi child → parent.

`waitpid()`

Menunggu child selesai dan ambil exit status.

`gettimeofday()`

Mengukur durasi eksekusi child.

`SIGINT`

Signal interupsi (Ctrl+C) untuk terminasi aman.

Zombie Process

Child selesai tapi entry di kernel belum dihapus karena parent belum `wait()`.

Laporan Tugas 1: Process Manager (NIM Ganjil / Shared Memory)

Laporan ini membahas implementasi Process Manager yang menggunakan forking untuk membuat proses anak dan Shared Memory sebagai mekanisme komunikasi (sesuai varian NIM Ganjil).

1. Diagram Process Tree

Tugas ini melibatkan satu proses induk (Parent) yang membuat tiga proses anak (Child) untuk menjalankan tugas yang berbeda secara bersamaan (concurrently).

Proses

Peran

Tugas yang Dikerjakan

Parent

Koordinator

Membuat 3 Child, menunggu (wait) semua Child selesai, dan menampilkan waktu eksekusi masing-masing.

Child 1

Pekerja

Menghitung Faktorial (Input: 5).

Child 2

Pekerja

Menghitung Bilangan Prima (Batas: 10).

Child 3

Pekerja

Membalik String (Input: "23").

2. Implementasi Kode dan Penjelasan System Call

Berikut adalah penjelasan fungsi utama dari system call yang digunakan dalam kode process_manager.c:

System Call

Mengapa Digunakan

Penjelasan

`fork()`

Digunakan untuk membuat proses baru (child process).

Setelah `fork()` dipanggil, program berjalan dalam dua proses yang identik. Nilai kembaliannya menentukan apakah kode selanjutnya dijalankan oleh Parent (`PID > 0`) atau Child (`PID == 0`). Kami memanggil `fork()` sebanyak tiga kali untuk membuat tiga Child.

`shmget()`

Digunakan untuk mengalokasikan segmen Shared Memory.

Kami meminta kernel untuk membuat atau mengakses segmen memori bersama. Ini diperlukan karena Child dan Parent perlu berbagi area memori yang sama untuk bertukar hasil (misalnya, hasil faktorial atau bilangan prima).

`shmat()`

Digunakan untuk menghubungkan segmen Shared Memory ke ruang alamat virtual proses.

Proses (Parent atau Child) harus memanggil ini agar benar-benar dapat membaca dan menulis data ke segmen memori bersama yang telah dialokasikan oleh `shmget()`.

`shmdt()`

Digunakan untuk melepaskan segmen Shared Memory dari ruang alamat proses.

Dipanggil setelah proses selesai menggunakan memori bersama untuk membersihkan koneksi, meskipun segmen memorinya sendiri masih ada di sistem.

`shmctl(IPC_RMID)`

Digunakan untuk menghapus segmen Shared Memory dari sistem.

Proses Parent harus memanggil ini setelah semua Child selesai dan hasilnya diambil, untuk menghindari sampah memori (memory leak) di sistem.

`wait()` / `waitpid()`

Digunakan oleh proses Parent untuk menunggu proses Child selesai.

Ini adalah kunci untuk sinkronisasi. Parent tidak boleh melanjutkan ke tugas berikutnya (misalnya, menampilkan hasil atau menghapus Shared Memory) sebelum Child selesai menjalankan tugas dan menulis hasilnya.

`sleep()`

Digunakan untuk menjeda eksekusi suatu proses selama beberapa detik.

Dalam tugas ini, `sleep()` digunakan untuk mensimulasikan waktu komputasi yang lama pada Child (misalnya, `sleep(2)` untuk Child 1) agar dapat diamati bahwa proses berjalan secara concurrent.

3. Konsep Zombie Process dan Cara Menghindarinya

Apa itu Zombie Process?

Zombie Process adalah proses anak yang telah selesai eksekusi tetapi entri tabel prosesnya masih ada di sistem karena proses induknya (Parent) belum memanggil system call `wait()` atau `waitpid()`.

Status: Proses berada dalam status Termination (Z di ps).

Masalah: Meskipun proses sudah mati dan tidak mengonsumsi memori utama, entri tabel prosesnya (menyimpan PID, status keluar, dll.) masih menempati slot di kernel. Jika banyak Zombie tercipta, ini dapat menghabiskan sumber daya kernel, meskipun dampaknya biasanya kecil.

Bagaimana Dihindari?

Untuk menghindari Zombie Process, proses Parent harus memanggil `wait()` atau `waitpid()` segera setelah proses Child dibuat dan Parent tidak memiliki pekerjaan lain, atau secara berkala.

Dalam kode Tugas 1, Zombie Process dihindari dengan cara:

Penggunaan `wait()`: Proses Parent secara eksplisit memanggil `wait()` atau `waitpid()` tiga kali (satu kali untuk setiap Child) dalam loop atau secara berurutan. Ini memastikan bahwa Parent "memanen" status keluar setiap Child, sehingga entri tabel prosesnya dihapus oleh kernel.

4. Pencatatan Error Selama Debugging

Selama proses pengembangan dan debugging kode `process_manager.c`, minimal dua jenis error umum berikut ditemui:

No.

Jenis Error Ditemui

Penjelasan Error dan Akibat

Solusi

1

stray '\302' in program

Ini adalah error kompilasi (gcc error), seperti yang terlihat pada tangkapan layar. Terjadi ketika karakter non-ASCII (seperti non-breaking space atau karakter yang disalin dari sumber web) masuk ke dalam kode. Akibatnya, kode sumber tidak bisa diubah menjadi program yang dapat dieksekusi.

Hapus dan Tulis Ulang Karakter: Menggunakan editor nano atau lainnya, karakter aneh tersebut harus dihapus total, dan diganti dengan karakter ASCII standar (misalnya, spasi atau semicolon yang benar).

2

Segmentation Fault (SIGSEGV) / Data Shared Memory Salah

Child Process atau Parent Process mencoba mengakses atau menulis ke area Shared Memory yang belum di-attach (`shmat`) atau sudah di-detach (`shmdt`). Akibatnya, program mengalami crash saat berjalan, atau data yang diambil Parent menjadi salah atau tidak valid.

Verifikasi Alamat dan Attach: Pastikan `shmat()` berhasil dipanggil sebelum proses mencoba menggunakan Shared Memory. Pastikan Parent tidak memanggil `shmdt()` atau `shmctl(IPC_RMID)` sebelum semua Child selesai dan data sudah diambil.

Laporan Tugas 2: Si Pemantau File (File Monitor) =Á

Tugas ini bertujuan membuat program yang terus berjalan (seperti penjaga) untuk memantau semua perubahan (pembuatan, modifikasi, penghapusan, dll.) yang terjadi pada file di dalam folder target. Program ini menggunakan system call khusus bernama `inotify`.

1. Fungsi Utama dan Perintah Kunci

Program ini bertindak sebagai daemon sederhana yang berjalan di latar belakang (hingga dihentikan paksa).

Perintah Utama yang Dieksekusi:

Bash

`./file_monitor /tmp/monitor_dir`

System Call Utama (inotify)

Untuk bisa mendeteksi perubahan, program menggunakan serangkaian perintah kernel:

System Call

Kenapa Dipakai (Penjelasan Manusiawi)

`inotify_init()`

Untuk Memulai Pemantauan. Ini seperti membuka telinga di kernel. Kita membuat file descriptor yang akan kita gunakan untuk mendaftarkan folder yang mau diawasi.

`inotify_add_watch()`

Untuk Menunjuk Folder yang Diawasi. Kita bilang ke kernel, "Tolong awasi folder /tmp/monitor_dir ini ya," dan kita tentukan jenis event apa saja yang kita minati (misalnya, `IN_CREATE`, `IN_DELETE`, `IN_MODIFY`).

`read()`

Untuk Mendengarkan Kejadian. Setelah mendaftarkan folder, program akan terus-menerus mencoba membaca dari file descriptor yang dibuat `inotify_init()`. Begitu ada event (misalnya file baru dibuat), kernel akan mengirim data yang kemudian kita baca dan kita proses.

2. Pencatatan Error Selama Debugging

Dalam pengembangan program yang melibatkan System Call yang berinteraksi langsung dengan kernel seperti `inotify`, ada error khas yang sering muncul. Berikut adalah dua error yang ditemui:

No.

Jenis Error Ditemui

Penjelasan Error dan Akibat

Solusi (Cara Mengatasi)

1

stray '\302' in program

Ini adalah Error Kompilasi yang muncul lagi (seperti di Tugas 1). Terjadi karena ada karakter non-ASCII (karakter asing, biasanya dari hasil copy-paste) di dalam kode sumber `file_monitor.c`. Akibatnya, program tidak bisa dikompilasi menjadi executable (`./file_monitor`).

Hapus dan Tulis Ulang Karakter: Buka kode sumber (nano file_monitor.c). Cari baris yang ditunjuk error, hapus karakter yang bermasalah, dan ketik ulang spasi, tanda kurung, atau titik koma yang seharusnya.

2

inotify_add_watch: Permission denied

Terjadi ketika program gagal mendaftarkan folder untuk diawasi karena tidak memiliki izin yang cukup. Biasanya terjadi jika kita mencoba memonitor folder milik sistem atau milik pengguna lain (misalnya, /root atau /usr/bin). Akibatnya, program tidak bisa mulai memantau dan langsung berhenti.

Pilih Direktori yang Tepat: Pastikan direktori yang dimonitor berada di lokasi yang dapat diakses penuh oleh pengguna yang menjalankan program (misalnya, di dalam home directory (~) atau di /tmp). Folder /tmp/monitor_dir yang kita gunakan adalah pilihan yang baik karena terbuka untuk diakses.

3

Loop Event Tak Terhingga (Pada Logging)

Kadang terjadi event berulang. Jika monitor mencatat event ke file log.txt dan file log.txt itu berada di dalam direktori yang dimonitor (/tmp/monitor_dir), maka aksi menulis ke log itu sendiri memicu event IN_MODIFY. Akibatnya, program akan crash atau log akan terisi tak terbatas (infinite loop) karena setiap event memicu event baru.

Pindahkan File Log: Simpan file log di luar direktori yang sedang dimonitor. Misalnya, jika memonitor /tmp/monitor_dir, simpan log di /var/log/monitor_saya.log

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.